1. Design a Pricing Plans Layout Page

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

1. Your page should have an `h1` element with the text `Pricing Plans`.
2. Your page should have a `div` element with the class `pricing-container` below the `h1` element.
- The `.pricing-container` selector should have a `display` property with the value of `flex` and a `flex-wrap` property with the value of `wrap`.
3. Within the `.pricing-container` element, you should have three `div` elements with the class `pricing-card` to represent the pricing plans.
- One of the `.pricing-card` elements should have the class `basic-plan` in addition to the `pricing-card` class.
- One of the `.pricing-card` elements should have the class `pro-plan` in addition to the `pricing-card` class.
- One of the `.pricing-card` elements should have the class `premium-plan` in addition to the `pricing-card` class.
4. Your `.basic-plan` element should have an `h2` element with the text `Basic`.
5. Your `.basic-plan` element should have a `p` element with the text `$9/month`.
6. Your `.pro-plan` element should have an `h2` element with the text `Pro`.
7. Your `.pro-plan` element should have a `p` element with the text `$19/month`.
8. Your `.premium-plan` element should have an `h2` element with the text `Premium`.
9. Your `.premium-plan` element should have a `p` element with the text `$29/month`.
10. Each of your `.pricing-card` elements should:
- Use Flexbox to stack its content in a column and justify the space between the children using `space-between`.
- Set the `flex` property to `0 0 200px` to give it a consistent width and prevent it from growing or shrinking in the layout.
- Set the `border` property to `2px solid black` to see how the different cards take up space.

11. The `.basic-plan` element should appear first in the layout. You should use the `order` property for this.
12. The `.pro-plan` element should appear second in the layout. You should use the `order` property and set its `flex-grow` property to `2` so it takes up more space than the other plans.
13. The `.premium-plan` element should come last in the layout. You should use the `order` property for this.

Steps

- 1. You should have an `h1` element.
- 2. Your page should have an `h1` element with the text `Pricing Plans`.
- 3. You should have a `div` element below the `h1` element.
- 4. Your `div` element should have the class `pricing-container`.
- 5. You should have a `.pricing-container` selector.
- 6. Your `pricing-container` selector should have the property `display` with a value of `flex`.
- 7. Your `.pricing-container` selector should have the property `flex-wrap` with a value of `wrap`.
- 8. You should have three `div` elements with the class `pricing-card` within the `.pricing-container` element.
- 9. One of your `.pricing-card` elements should have the class `basic-plan`.
- 10. One of your `.pricing-card` elements should have the class `premium-plan`.
- 11. One of your `.pricing-card` elements should have the class `pro-plan`.
- 12. You should have a `.pricing-card` selector.
- 13. Your `.pricing-card` selector should have the property `display` with a value of `flex`.
- 14. Your `.pricing-card` selector should have the property `flex-direction` with a value of `column`.
- 15. Your `.pricing-card` selector should have the property `justify-content` with a value of `space-between`.
- 16. Your `.pricing-card` selector should have the property `flex` with a value of `0 0 200px`.
- 17. Your `.pricing-card` selector should have the property `border` with a value of `2px solid black`.

- 18. Your `.basic-plan` selector should have the property `order` with a value of `0`.
- 19. Your `.basic-plan` element should have an `h2` element with the text `Basic`.
- 20. Your `.basic-plan` element should have a `p` element with the text `$9/month`.
- 21. Your `.pro-plan` element should have an `h2` element with the text `Pro`.
- 22. Your `.pro-plan` element should have a `p` element with the text `$19/month`.
- 23. Your `.pro-plan` element should have the property `order` with a value of `1`.
- 24. Your `.pro-plan` element should have the property `flex-grow` with a value of `2`.
- 25. Your `.premium-plan` element should have an `h2` element with the text `Premium`.
- 26. Your `.premium-plan` element should have a `p` element with the text `$29/month`.
- 27. Your `premium-plan` element should be the last element in the layout and have the property `order` and the value `2`.

2. Build a Page of Playing Cards

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

Objective: Fulfill the user stories below and get all the tests to pass to complete the lab.

User Stories:

1. You should build a webpage that displays at least three playing cards.
2. You should have a `main` element with an ID of `playing-cards`.
3. Within your `#playing-cards` element, you should have at least three `div` elements, each with a class of `card`.
4. Within each `.card` element, you should have three `div` elements, the first with a class of `left`, the second with a class of `middle`, and the third with a class of `right`.
5. Your `#playing-cards` element should use flexbox to horizontally center its children, allow them to wrap, and put a `20px` space between them.

6. Each of your `.card` elements should use flexbox to justify its children using `space-between`.
7. Each of your `.left` elements should use flexbox item properties to align itself at the start of its' parent container.
8. Each of your `.middle` elements should use flexbox item properties to align itself in the center of its' parent container.
9. Each of your `.right` elements should use flexbox item properties to align itself at the end of its parent container.
10. Each of your `.middle` elements should use flexbox to display its children in a column.

Steps:

- 1. You should have a `main` element with an `id` of `playing-cards`.
- 2. You should have at least three `div` elements with a class of `card` within your `#playing-cards` element.
- 3. Each of your `.card` elements should have a width and height.
- 4. Each of your `.card` elements should have exactly three `div` elements as children.
- 5. You should have a `div` with a class of `left` within each `.card` element.
- 6. You should have a `div` with a class of `middle` within each `.card` element.
- 7. You should have a `div` with a class of `right` within each `.card` element.
- 8. You should have a `#playing-cards` selector that sets its elements' `display` to `flex`.
- 9. You should have a `#playing-cards` selector that sets its elements' `justify-content` to `center`.
- 10. You should have a `#playing-cards` selector that sets its elements' `flex-wrap` to `wrap`.
- 11. You should have a `#playing-cards` selector that sets its elements' `gap` to `20px`.
- 12. You should have a `.card` selector that sets its elements' `display` to `flex`.
- 13. You should have a `.card` selector that sets its elements' `justify-content` to `space-between`.
- 14. You should have a `.left` selector that sets its elements' `align-self` to `flex-start`.
- 15. You should have a `.middle` selector that sets its elements' `align-self` to `center`.

- 16. You should have a `.right` selector that sets its elements' `align-self` to `flex-end`.
- 17. You should have a `.middle` selector that sets its elements' `display` to `flex`.
- 18. You should have a `flex-direction` set to `column` in your `.middle` class.

3. Build a Technical Documentation Page

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

1. You can see a `main` element with a corresponding `id="main-doc"`, which contains the page's main content (technical documentation).
2. Within the `#main-doc` element, you can see several `section` elements, each with a class of `main-section`. There should be a minimum of five.
3. The first element within each `.main-section` should be a `header` element, which contains text that describes the topic of that section.
4. Each `section` element with the class of `main-section` should also have an `id` that corresponds with the text of each `header` contained within it. Any spaces should be replaced with underscores (e.g. The section that contains the header "JavaScript and Java" should have a corresponding `id="JavaScript_and_Java"`).
5. The `.main-section` elements should contain at least ten `p` elements total (not each).
6. The `.main-section` elements should contain at least five `code` elements total (not each).
7. The `.main-section` elements should contain at least five `li` items total (not each).
8. You can see a `nav` element with a corresponding `id="navbar"`.
9. The navbar element should contain one `header` element which contains text that describes the topic of the technical documentation.
10. Additionally, the navbar should contain link (`a`) elements with the class of `nav-link`. There should be one for every element with the class `main-section`.
11. The `header` element in the `#navbar` must come before any link (`a`) elements in the navbar.

12. Each element with the class of `nav-link` should contain text that corresponds to the `header` text within each `section` (e.g. if you have a "Hello world" section/header, your navbar should have an element which contains the text "Hello world").
13. When you click on a navbar element, the page should navigate to the corresponding section of the `#main-doc` element (e.g. If you click on a `.nav-link` element that contains the text "Hello world", the page navigates to a `section` element with that id, and contains the corresponding header).
14. On regular sized devices (laptops, desktops), the element with `id="navbar"` should be shown on the left side of the screen and should always be visible to the user.
15. Your technical documentation should use at least one media query.

Steps:

- 1. You should have a `main` element with an `id` of `main-doc`.
- 2. You should have at least five `section` elements with a class of `main-section`.
- 3. All of your `.main-section` elements should be `section` elements.
- 4. You should have at least five `.main-section` elements that are descendants of `#main-doc`.
- 5. The first child of each `.main-section` should be a `header` element.
- 6. None of your `header` elements should be empty.
- 7. All of your `.main-section` elements should have an `id`.
- 8. Each `.main-section` should have an `id` that matches the text of its first child, having any spaces in the child's text replaced with underscores (`_`) for the id's.
- 9. You should have at least 10 `p` elements (total) within your `.main-section` elements.
- 10. You should have at least five `code` elements that are descendants of `.main-section` elements.
- 11. You should have at least five `li` elements that are descendants of `.main-section` elements.
- 12. You should have a `nav` element with an `id` of `navbar`.
- 13. Your `#navbar` should have exactly one `header` element within it.
- 14. You should have at least one `a` element with a class of `nav-link`.
- 15. All of your `.nav-link` elements should be anchor (`a`) elements.
- 16. All of your `.nav-link` elements should be in the `#navbar`.

- 17. You should have the same number of `.nav-link` and `.main-section` elements.
- 18. The `header` element in the `#navbar` should come before any link (`a`) elements also in the `#navbar`.
- 19. Each `.nav-link` should have text that corresponds to the `header` text of its related `section` (e.g. if you have a "Hello world" section/header, your `#navbar` should have a `.nav-link` which has the text "Hello world").
- 20. Each `.nav-link` should have an `href` attribute that links to its corresponding `.main-section` (e.g. If you click on a `.nav-link` element that contains the text "Hello world", the page navigates to a `section` element with that id).
- 21. Your `#navbar` should always be on the left edge of the window.
- 22. Your Technical Documentation project should use at least one media query.

4. A Newspaper Layout

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

In this lab, you will design a newspaper layout using CSS Grid, including concepts like grid rows and grid columns.

Objective: Fulfill the user stories below and get all the tests to pass to complete the lab.

User Stories:

1. Your page should contain a `main` element with the class `newspaper-layout`.
2. The `.newspaper-layout` should include a `header` with the class `title` containing an `h1` to display the newspaper's name.
3. The `.newspaper-layout` should include an `article` with the class `feature-article` for the main news article.
4. The `.feature-article` should include an `h2` element for the article title followed by a `p` element for the article content.
5. You should add three more `article` elements for smaller articles, with classes `small-article1`, `small-article2`, and `small-article3`.

6. Each of the smaller articles should include an `h3` element for the article title followed by a `p` element for the article content.
7. The `.newspaper-layout` should include an `article` element with the class `secondary-article` for an additional news section.
8. The `.secondary-article` should include an `h2` element for the article title followed by a `p` element for the article content.
9. The `.newspaper-layout` should include a `figure` with the class `cover-image` to display an image that represents the newspaper's content.
10. The elements with the classes `title`, `feature-article`, `secondary-article`, `cover-image`, `small-article1`, `small-article2`, and `small-article3` should have a `grid-area` property set to the same class name.
11. Your `.newspaper-layout` should use CSS Grid with a `grid-template-areas` property to define the arrangement of sections:

- The `.title` should span across the top row.
- The `.feature-article` should span two columns in the second row, with the `.cover-image` in the third column.
- The `.secondary-article` should span two columns in the third row, with the `.cover-image` in the third column.
- The three small articles should appear in the fourth row.

12. The `.newspaper-layout` should have a `grid-template-columns` property that divides the space into three equal columns.
13. You should set the `.newspaper-layout`'s `grid-template-rows` property to `auto` for the first row and divide the remaining space into equal parts for the other rows.
14. You should add a gap between grid items.
15. Ensure that the image inside `.cover-image` fits within the designated space by setting its `max-width` to `100%`.

Note: Be sure to link your stylesheet in your HTML and apply your CSS.

Steps:

- 1. You should have a `main` element.
- 2. Your `main` element should have a `class` of `newspaper-layout`.
- 3. You should have a `header` element with the `class` of `title` within your `.newspaper-layout` element.
- 4. Your `header` element should contain an `h1` element.
- 5. Your `.title` element should contain some text.

- 6. You should have an `article` element for each article with classes `feature-article`, `small-article1`, `small-article2`, `small-article3`, and `secondary-article`, respectively.
- 7. The `.feature-article` should include an `h2` element followed by a `p` element.
- 8. `.small-article1`, `.small-article2`, and `.small-article3` should include an `h3` element followed by a `p` element.
- 9. The `.secondary-article` should include an `h2` element followed by a `p` element.
- 10. You should have a `figure` element with the class of `cover-image` within your `.newspaper-layout`.
- 11. Your `.cover-image` should contain an `img` element.
- 12. The `display` property of the `.newspaper-layout` element should be set to `grid`.
- 13. Your `.newspaper-layout` element should have a `grid-template-areas` property with `title` spanning all three columns of the first row.
- 14. The second row of the grid template should contain a `feature-article` spanning two columns and a `cover-image`.
- 15. The third row of the grid template should contain a `secondary-article` spanning two columns and a `cover-image`.
- 16. The fourth row of the grid template should contain `small-article1`, `small-article2`, and `small-article3`.
- 17. Your `.newspaper-layout` selector should use the `grid-template-columns` property to divide the space into three equal parts. Remember you can use either the `repeat` function or manually define each fractional unit.
- 18. Your `.newspaper-layout` selector should have a `grid-template-rows` property set to `auto 1fr 1fr 1fr`.
- 19. You should add a gap between grid items.
- 20. `.title` should have a `grid-area` of `title`.
- 21. `.feature-article` should have a `grid-area` of `feature-article`.
- 22. `.cover-image` should have a `grid-area` of `cover-image`.
- 23. `.secondary-article` should have a `grid-area` of `secondary-article`.
- 24. `.small-article1` should have a `grid-area` of `small-article1`.
- 25. `.small-article2` should have a `grid-area` of `small-article2`.
- 26. `.small-article3` should have a `grid-area` of `small-article3`.
- 27. The `img` inside `.cover-image` should have a `max-width` of `100%`.

5. Build a Product Landing Page

Build an app that is functionally similar to this example project. Try not to copy the example project, give it your own personal style.

Objective: Fulfill the user stories below and get all the tests to pass to complete the lab.

User Stories:

1. Your product landing page should have a `header` element with a corresponding `id="header"`.
2. You should have an image within the `header` element with a corresponding `id="header-img"` (A logo would make a good image here).
3. Within the `#header` element, you should have a `nav` element with a corresponding `id="nav-bar"`.
4. You should have at least three clickable elements inside the `nav` element, each with the class `nav-link`.
5. When you click a `.nav-link` button in the `nav` element, you should be taken to the corresponding section of the landing page.
6. You should have an embedded product video with `id="video"`.
7. Your landing page should have a `form` element with a corresponding `id="form"`.
8. Within the form, there should be an `input` field with `id="email"` where you can enter an email address.
9. The `#email` input field should have placeholder text to let users know what the field is for.
10. The `#email` input field should use HTML5 validation to confirm that the entered text is an email address.
11. Within the form, there should be a submit `input` with a corresponding `id="submit"`.
12. When you click the `#submit` element, the email should be submitted to a static page (use this mock URL: `https://www.freecodecamp.org/email-submit`).
13. The navbar should always be at the top of the viewport.
14. Your product landing page should have at least one media query.
15. Your product landing page should utilize CSS flexbox at least once.

Steps:

- 1. You should have a `header` element with an `id` of `header`.

- 2. You should have an `img` element with an `id` of `header-img`.
- 3. Your `#header-img` should be a descendant of the `#header`.
- 4. Your `#header-img` should have a `src` attribute.
- 5. Your `#header-img`'s `src` value should be a valid URL (starts with `http`).
- 6. You should have a `nav` element with an `id` of `nav-bar`.
- 7. Your `#nav-bar` should be a descendant of the `#header`.
- 8. You should have at least 3 `.nav-link` elements within the `#nav-bar`.
- 9. Each `.nav-link` element should have an `href` attribute.
- 10. Each `.nav-link` element should link to a corresponding element on the landing page (has an `href` with a value of another element's id. e.g. `#footer`).
- 11. You should have a `video` or `iframe` element with an `id` of `video`.
- 12. Your `#video` should have a `src` attribute.
- 13. You should have a `form` element with an `id` of `form`.
- 14. You should have an `input` element with an `id` of `email`.
- 15. Your `#email` should be a descendant of the `#form`.
- 16. Your `#email` should have the `placeholder` attribute with placeholder text.
- 17. Your `#email` should use HTML5 validation by setting its `type` to `email`.
- 18. You should have an `input` element with an `id` of `submit`.
- 19. Your `#submit` should be a descendant of the `#form`.
- 20. Your `#submit` should have a `type` of `submit`.
- 21. Your `#form` should have an `action` attribute of `https://www.freecodecamp.org/email-submit`.
- 22. Your `#email` should have a `name` attribute of `email`.
- 23. Your `#nav-bar` should always be at the top of the viewport.
- 24. Your Product Landing Page should use at least one media query.
- 25. Your Product Landing Page should use CSS Flexbox at least once.

Question 1 - Solution

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Art Studio - Pricing Plans</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            overflow-x: hidden;
        }
        .split-container {
            display: flex;
            min-height: 100vh;
        }
        .hero-section {
            flex: 1;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            padding: 60px;
            color: white;
            display: flex;
            flex-direction: column;
            justify-content: center;
        }
        .hero-title {
            font-size: 4em;
            margin-bottom: 15px;
            text-shadow: 2px 2px 4px rgba(0,0,0,0.2);
        }
        .hero-subtitle {
            font-size: 1.5em;
            opacity: 0.95;
            margin-bottom: 40px;
        }
        .art-grid {
            display: grid;
            grid-template-columns: repeat(2, 1fr);
            gap: 20px;
            margin-bottom: 30px;
        }
        .art-item {
            aspect-ratio: 1;
            border-radius: 15px;
            overflow: hidden;
            box-shadow: 0 10px 30px rgba(0,0,0,0.3);
            transition: transform 0.3s ease;
        }
        .art-item:hover {
            transform: scale(1.05);
        }
        .art-item img {
            width: 100%;
            height: 100%;
            object-fit: cover;
        }
```

Question 2 - Solution

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Poker Hand - Playing Cards</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            background: linear-gradient(135deg, #0F2027 0%, #203A43 50%, #2C5364 100%);
            font-family: 'Arial', sans-serif;
            padding: 50px 20px;
            min-height: 100vh;
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
        }
        h1 {
            color: white;
            font-size: 3.5em;
            text-align: center;
            margin-bottom: 20px;
            text-shadow: 2px 2px 4px rgba(0,0,0,0.5);
        }
        .subtitle {
            color: rgba(255,255,255,0.8);
            font-size: 1.3em;
            text-align: center;
            margin-bottom: 50px;
        }
        #playing-cards {
            display: flex;
            gap: 20px;
            flex-wrap: wrap;
            justify-content: center;
            perspective: 1000px;
        }
        .card {
            width: 180px;
            height: 250px;
            background: white;
            border-radius: 15px;
            padding: 20px;
            display: flex;
            flex-direction: column;
            justify-content: space-between;
            box-shadow: 0 15px 40px rgba(0,0,0,0.5);
            transition: all 0.3s ease;
            cursor: pointer;
            position: relative;
        }
        .card:hover {
            transform: translateY(-20px) rotateY(10deg);
            box-shadow: 0 25px 60px rgba(0,0,0,0.6);
        }
```

Question 3 - Solution

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>C++ Documentation</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: #f5f7fa;
            display: flex;
            min-height: 100vh;
        }
        #navbar {
            width: 300px;
            background: linear-gradient(180deg, #1e3c72 0%, #2a5298 100%);
            color: white;
            position: fixed;
            height: 100vh;
            overflow-y: auto;
            box-shadow: 3px 0 10px rgba(0,0,0,0.1);
        }
        #navbar header {
            font-size: 2em;
            padding: 30px 20px;
            border-bottom: 2px solid rgba(255,255,255,0.2);
            font-weight: bold;
        }
        .nav-link {
            display: block;
            color: rgba(255,255,255,0.9);
            text-decoration: none;
            padding: 15px 20px;
            border-left: 4px solid transparent;
            transition: all 0.3s ease;
        }
        .nav-link:hover {
            background: rgba(255,255,255,0.1);
            border-left-color: #4fc3f7;
            padding-left: 25px;
        }
        #main-doc {
            margin-left: 300px;
            padding: 50px;
            flex: 1;
        }
        .main-section {
            background: white;
            border-radius: 15px;
            padding: 40px;
            margin-bottom: 30px;
            box-shadow: 0 5px 20px rgba(0,0,0,0.08);
        }
        .main-section header {
            font-size: 2.5em;
```

Question 4 - Solution

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>The Daily Tech Tribune</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: 'Georgia', serif;
            background: #f5f5f5;
            padding: 30px;
        }
        .newspaper-layout {
            max-width: 1200px;
            margin: 0 auto;
            background: white;
            padding: 40px;
            box-shadow: 0 10px 40px rgba(0,0,0,0.1);
        }
        .title {
            text-align: center;
            border-bottom: 4px double #2c3e50;
            padding-bottom: 20px;
            margin-bottom: 30px;
        }
        .title h1 {
            font-size: 4em;
            font-family: 'Times New Roman', serif;
            color: #2c3e50;
            letter-spacing: 2px;
            margin-bottom: 10px;
        }
        .subtitle {
            color: #7f8c8d;
            font-size: 1.1em;
            font-style: italic;
        }
        article h2 {
            font-size: 2em;
            color: #2c3e50;
            margin-bottom: 15px;
            line-height: 1.3;
        }
        article p {
            line-height: 1.8;
            color: #34495e;
            margin-bottom: 15px;
            text-align: justify;
            font-size: 1.05em;
        }
        .feature-article {
            column-count: 2;
            column-gap: 40px;
            margin-bottom: 30px;
            padding-bottom: 30px;
```

Question 5 - Solution

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>TechWatch Pro - Your Smart Companion</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: #fff;
        }
        #header {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            padding: 20px 50px;
            position: sticky;
            top: 0;
            z-index: 1000;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        }
        .header-container {
            max-width: 1200px;
            margin: 0 auto;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }
        #header-img {
            height: 50px;
        }
        #nav-bar {
            display: flex;
            gap: 30px;
        }
        .nav-link {
            color: white;
            text-decoration: none;
            font-size: 1.1em;
            transition: all 0.3s ease;
            padding: 8px 16px;
            border-radius: 5px;
        }
        .nav-link:hover {
            background: rgba(255,255,255,0.2);
        }
        #hero {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            text-align: center;
            padding: 100px 20px;
        }
        #hero h1 {
            font-size: 4em;
            margin-bottom: 20px;
            text-shadow: 2px 2px 4px rgba(0,0,0,0.2);
```